

fragmento que vamos a usar:

No tienes que ser buena.
No tienes que recorrer el desierto de rodillas, arrepintiéndote.
Sólo deja que el suave animal de tu cuerpo ame lo que ama.
Háblame del dolor, del tuyo, yo te hablaré del mío.
Mientras tanto, el mundo sigue.

licencia

El poema está bajo la licencia de copyright, donde todos los derechos están reservados, pero se puede utilizar bajo el concepto legal del uso legítimo, ya que es sin fines de lucro y con propósitos educativos.

"Proyecto académico / Fragmento de 'Wild Geese' por Mary Oliver / Uso educativo no comercial"

© Wild Geese, 1986 Mary Oliver

animación

video de referencia:

<https://pixabay.com/es/videos/ganso-salvaje-aterrizaje-de-gansos-342468/>

¿qué queremos hacer?**prompt para estructura:**

crea estructura conceptual del siguiente flujo para trabajar arduino con pantalla pantalla OLED I2C SSD1306, con potenciómetro y botón.

La estructura debe entenderse como una secuencia de interacción progresiva. Al iniciar el sistema, la pantalla muestra primero un texto de licencia y una pequeña animación. Después aparece un aviso que invita a presionar el botón. Con esa primera presión se presenta el título del poema y cada nueva presión del botón permite avanzar verso por verso: la primera muestra el verso 01, la segunda el verso 02, la tercera el verso 03, la cuarta el verso 04 y la quinta el verso 05.

Solo una vez que se ha mostrado el quinto verso, y por lo tanto después de completar las cinco presiones correspondientes a los versos, la pantalla se limpia y se habilita el uso del potenciómetro. Aunque este se encuentre conectado físicamente desde el inicio, el programa debe ignorar su lectura hasta llegar a esta etapa. Al mover el potenciómetro, el nombre de la autora debe revelarse progresivamente, letra por letra, según la posición del control. Finalmente, cuando el sistema se encuentre en esta última etapa, una doble presión rápida del botón debe reiniciar toda la experiencia y volver al texto de licencia inicial.

respuesta chat gpt:

botón



título

↓
botón 01 → verso[0]
↓
botón 02 → verso[1]
↓
botón 03 → verso[2]
↓
botón 04 → verso[3]
↓
botón 05 → verso[4]

Después del verso 5, el botón **deja de servir para avanzar contenido** y cambia su función. Ahora:

potenciómetro
↓
controla cuántas letras
de la autora aparecen

mientras que:

botón + botón rápidamente
↓
reiniciar experiencia

prompt para estructura:

Según mis apuntes, quiero ordenar correctamente la estructura del código para Arduino usando char. Entiendo que primero debería incluir las librerías necesarias para la pantalla OLED, luego configurar la pantalla en el código y definir las conexiones del botón y del potenciómetro. Después debería declarar los textos que utilizará el proyecto, como el texto de licencia, el título del poema, los versos y el nombre de la autora, pero no tengo claro en qué momento conviene declarar las variables, crear funciones o si realmente es necesario utilizar una clase.

La interacción debería funcionar así: al iniciar, aparece el texto de licencia; luego se ejecuta una pequeña animación; después aparece un mensaje de "Presiona el botón" que parpadea. Al presionar el botón se muestra el título del poema y, desde ahí, cada nueva presión muestra uno de los cinco versos, almacenados en un arreglo de char. Una vez mostrado el quinto verso, la pantalla se limpia y recién en ese momento se habilita el potenciómetro. Al moverlo, debería revelarse progresivamente, letra por letra, el nombre de la autora, utilizando el valor del potenciómetro para determinar cuántas letras mostrar y un for para recorrer los caracteres correspondientes. Finalmente, al detectar una doble presión del botón, todo el sistema debería reiniciarse y volver al texto de licencia inicial.

Necesito ayuda para ordenar esta lógica en la estructura correcta de un programa de Arduino: qué debe ir primero, qué corresponde a variables, qué debería resolverse con funciones, si hace falta una clase y cómo organizar después `setup()` y `loop()`.

respuesta chat gpt:

Para ordenar bien el código, conviene pensar primero en qué cosas existen durante todo el programa y después en qué cosas ocurren durante la ejecución. Al comienzo deberían ir las librerías necesarias para trabajar con la pantalla OLED, como `Wire`, `Adafruit_GFX` y `Adafruit_SSD1306`. Inmediatamente después se configura el objeto `display` que representa la pantalla y se declaran los pines que usarán el botón y el potenciómetro. Esa primera parte del archivo sirve para definir el hardware con el que trabajará Arduino.

Después deberían declararse todos los textos que el programa necesita tener disponibles desde el inicio. Como quieres trabajar con `char`, aquí podrías guardar el texto de licencia, el título del poema, el arreglo con los cinco versos y el nombre de la autora. Los versos tendrían sentido como un arreglo del tipo `const char *versos[ ]`, porque necesitas acceder a cada uno individualmente según la cantidad de veces que se presione el botón. En cambio, el título, la licencia y el nombre de la autora pueden ser cadenas individuales de caracteres.

Luego vendrían las variables que sirven para recordar qué está pasando durante la experiencia. Por ejemplo, una variable `versoActual` puede guardar si estás en el verso 0, 1, 2, 3 o 4. También necesitas alguna variable que indique en qué etapa del programa estás, porque el botón no hace siempre lo mismo. Al principio sirve para mostrar el título, después sirve para avanzar entre versos y, al final, sirve para detectar el doble clic que reinicia el sistema. También necesitarás variables para recordar el estado anterior del botón, el valor leído por el potenciómetro y cuántas letras de la autora deben aparecer.

En este proyecto no parece necesario crear una clase propia. Una clase tendría más sentido si quisieras diseñar algo como un objeto `Poema` que tuviera internamente sus versos, animaciones y comportamientos. Para lo que estás haciendo ahora, usar variables, arreglos y funciones es más claro y te permite entender mejor la lógica.

Después de las variables conviene crear funciones para separar cada comportamiento. Podrías tener una función que muestre la licencia, otra que ejecute la pequeña animación, otra que muestre el aviso parpadeante de "Presiona el botón", otra que muestre el título, otra que muestre el verso correspondiente, otra que maneje la aparición de la autora y otra que reinicie la experiencia. La ventaja es que luego el `loop()` no queda lleno de instrucciones de la pantalla, sino que puede concentrarse en decidir qué función corresponde ejecutar.

El `setup()` debería encargarse solamente de preparar el sistema cuando Arduino se enciende. Ahí se configura el botón con `pinMode`, se inicia la pantalla OLED, se limpia y se establecen propiedades generales como el color y el tamaño del texto. El potenciómetro

también queda físicamente conectado desde el principio, aunque eso no significa que tengas que usar su lectura desde el comienzo.

El `loop()` sería el encargado de controlar toda la secuencia. Primero se muestra la licencia y luego la animación. Después aparece el mensaje de “Presiona el botón”, que puede parpadear mientras Arduino espera una presión. Cuando ocurre esa primera presión, se muestra el título del poema. A partir de ahí se entra en la etapa de los versos. Cada nueva presión muestra `versos[versoActual]` y después aumenta `versoActual` usando `versoActual++`. Mientras `versoActual` sea menor que 5, el programa sigue esperando nuevas presiones.

Cuando se ha mostrado el quinto verso, el programa cambia de etapa. La pantalla se limpia y recién desde ese momento comienza a utilizarse la lectura del potenciómetro. El potenciómetro entrega un valor numérico mediante `analogRead()`, y ese valor se transforma en una cantidad de letras que deben aparecer. El `for` no controla el potenciómetro: el potenciómetro decide cuántas letras se muestran y el `for` recorre el nombre de la autora desde la primera letra hasta esa cantidad. Por ejemplo, si el valor del potenciómetro corresponde a cuatro letras, el `for` recorre `autora[0]`, `autora[1]`, `autora[2]` y `autora[3]`, mostrando “Mary”. Si se mueve más, el `for` recorre más posiciones y el nombre continúa apareciendo.

Finalmente, en esta última etapa el botón cambia nuevamente de función. Ya no avanza versos, sino que se utiliza para detectar dos presiones rápidas. Si se reconoce el doble clic, una función de reinicio devuelve `versoActual` a 0, cambia la etapa nuevamente al inicio, limpia la OLED y toda la experiencia comienza otra vez desde el texto de licencia.

Por lo tanto, la estructura general sería conceptualmente: primero librerías y configuración de hardware, después textos en `char`, luego variables de estado, después funciones, luego `setup()` y finalmente `loop()`. Dentro del `loop()` lo más importante no será escribir todo de corrido, sino ir preguntando en qué etapa se encuentra el programa y actuar según esa etapa. Esa es la lógica que va a permitir que el mismo botón y el mismo potenciómetro tengan comportamientos diferentes dependiendo de dónde se encuentre el usuario en la experiencia.

Conexiones:

Pantalla	Arduino
GND	GND
VCC	5V
SCK/SCL	A4
SDA	A5

***push pul= lógica false para botón**

lógica invertida por INPUT_PULLUP

https://cursos.mcielectronics.cl/2024/10/22/explicando-el-modo-arduino-input_pullup-pinmode/

botón SIN presionar → HIGH → true botón PRESIONADO → LOW → false

probar conexión botón led: <https://www.youtube.com/watch?v=vxz2oYDYawQ&t=538s>

<https://www.tinkercad.com/things/i68pVaEyGKI-03-encender-y-apagar-led-en-board-con-pulsador>

para boton: <https://docs.arduino.cc/built-in-examples/digital/Button/>

declarar que tenemos botón código:

```
const int buttonPin = 12; // the number of the pushbutton pin
const int ledPin = 13; // the number of the LED pin
```

revisar para armar el flujo(? : mermaid language

potenciometro:

```
const int potenciometro = A0;

void setup() {
  Serial.begin(9600);
}

void loop() {
  int valorPot = analogRead(potenciometro);

  Serial.print("Valor: ");
  Serial.println(valorPot);

  delay(100);
}
```

declarar potenciometro:

```
const int potenciometro = A0;
```

<https://docs.arduino.cc/learn/electronics/potentiometer-basics/>

```
const int potenciometro = A0;
const int buttonPin = 12; // pushbutton pin
const int ledPin = 13; // LED pin
int buttonState = 0;

void setup() {

  pinMode(ledPin, OUTPUT);
  pinMode(buttonPin, INPUT);
  Serial.begin(9600);
}

void loop() {
  int valorPot = analogRead(potenciometro);

  // potenciometro
  Serial.print("Valor Pote: ");
  Serial.println(valorPot);

  // botón y led
  delay(100);

  // read the state of the pushbutton value:
  buttonState = digitalRead(buttonPin);

  // check if the pushbutton is pressed. If it is, the buttonState is HIGH:
  if (buttonState == HIGH) {
    // turn LED on:
    digitalWrite(ledPin, HIGH);
    Serial.println("VALOR boton ");
    Serial.println(buttonState);
  } else {
    // turn LED off:
    digitalWrite(ledPin, LOW);
    Serial.println("VALOR boton ");
    Serial.println(buttonState);
  }
}
```

duuudasss: no estoy cachando xq en el código no se declara el pin de la pantalla así que voy a dejar anotada la pregunta y la respuesta para el proceso... jejeje

La pantalla se comunica con Arduino mediante I2C, que utiliza los pines SDA y SCL. El programa sabe cuáles son esos pines gracias a la librería `Wire` y a la configuración de la placa que tengo seleccionada en Arduino. Por eso, cuando en el código aparece `&Wire`, no es necesario escribir directamente A4 o A5, ya que Arduino ya sabe qué pines corresponden a SDA y SCL en esa placa.

En el Arduino UNO R4 WiFi, la pantalla se conecta de esta forma:

- `SDA` de la pantalla → `SDA` / `A4`
- `SCL` de la pantalla → `SCL` / `A5`
- `VCC` → alimentación de la pantalla
- `GND` → `GND`

Por otro lado, la librería `Adafruit_SSD1306` permite controlar específicamente la pantalla, por ejemplo para escribir texto, limpiar la pantalla o actualizar lo que se muestra.

Finalmente, `0x3C` no corresponde a un pin, sino a la dirección I2C de la pantalla. Entonces, `Wire` permite la comunicación por SDA y SCL, mientras que `0x3C` identifica específicamente a la pantalla dentro de esa comunicación.

None

```
#include <Wire.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 32
#define OLED_RESET -1
#define SCREEN_ADDRESS 0x3C

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT,
&Wire, OLED_RESET);
```

1er paso fue hacer funcionar cada componente de forma individual,,, primera parte del código entonces es la declaración de: botón, led, pantalla, potenciómetro

//Bibliotecas que vamos a usar, es principalmente importante ya que con esta hacemos el llamado a la pantalla oled

```
#include <Wire.h>
```

```
#include <Adafruit_GFX.h>
```

```
#include <Adafruit_SSD1306.h>
```

// Pantalla OLED aquí declaramos la pantalla y sus características

```
#define SCREEN_WIDTH 128
```

```
#define SCREEN_HEIGHT 32
```

```
#define OLED_RESET -1
```

```
#define SCREEN_ADDRESS 0x3C
```

// Tipo de objeto

```
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire,  
OLED_RESET);
```

// Componentes, importante revisar que los pin estén correctos

```
const int potenciometro = A0;
```

```
const int buttonPin = 12; // pushbutton pin
```

```
const int ledPin = 13; // LED pin
```

otra duda es xq no llevan ; los define= **#define** no es una instrucción normal de C++, sino una directiva del preprocesador. Básicamente le dice al programa: “cada vez que veas **SCREEN_WIDTH**, reemplázalo por **128**”

el Seba nos dijo q probamos todos los componentes que funcionen por si solos y si funcionan asi que eso cheeekkkkk

[illegible]

[illegible][illegible]

[illegible][illegible]

```
0xff, 0xff, 0xff, 0xff, 0xff, 0xf0, 0x1f, 0xff, 0xff, 0xff, 0xff, 0xff, 0xfe, 0x08,  
0xff, 0xff, 0xff, 0xff, 0xff, 0xf8, 0x1f, 0xff, 0xff, 0xff, 0xff, 0xff, 0x0e,  
0xff, 0xff, 0xff, 0xff, 0xff, 0xfe, 0x1f, 0xff, 0xff, 0xff, 0xff, 0xff, 0x2f,  
0xff, 0xff, 0xff, 0xff, 0xff, 0x0f, 0xff, 0xff, 0xff, 0xff, 0xff, 0x1f  
};
```

```
// Array que reúne los 6 frames  
const int epd_bitmap_allArray_LEN = 6;
```

```
const unsigned char* epd_bitmap_allArray[6] = {  
    epd_bitmap_frames_animacion_1,  
    epd_bitmap_frames_animacion_2,  
    epd_bitmap_frames_animacion_3,  
    epd_bitmap_frames_animacion_4,  
    epd_bitmap_frames_animacion_5,  
    epd_bitmap_frames_animacion_6  
};
```

3er paso declarar poema y versos y texto general:

declarar todos los textos con el código que nos enseñó aarón

```
// TEXTOS DEL PROYECTO
```

```
const char* versos[] = {  
    "No tienes",  
    "que ser buena",  
    "No tienes",  
    "que recorrer el desierto",  
    "de rodillas", "arrepintiendote"  
};
```

corregimos este ya que no iba a entrar en la pantalla

```
const char* partesVersos[] = {  
  
    // Verso 1  
    "No tienes\nque ser buena.",  
  
    // Verso 2 - parte 1  
    "No tienes que\nrecorrer el\ndesierto de",  
  
    // Verso 2 - parte 2  
    "rodillas,\narrepintiendote.",  
  
    // Verso 3 - parte 1  
    "Solo deja que\nel suave animal",  
  
    // Verso 3 - parte 2  
    "de tu cuerpo\name lo que ama.",  
  
    // Verso 4  
    "Hablame del dolor,\ndel tuyo,\nyo te hablare del mio.",  
  
    // Verso 5  
    "Mientras tanto,\nel mundo sigue."  
};
```

```
const char autora[] = "Mary Oliver";
const char titulo[] = "Gansos salvajes";
const char instruccionPot[] = "Gira potenciometro";
```

```
const char* licencia[] = {
    "Proyecto academico",
    "Fragmento de 'Wild Geese'",
    "por Mary Oliver",
    "Uso educativo no comercial",
    "Wild Geese, 1986",
    "Mary Oliver"
};
```

```
const char* mensajeBoton[] = {
    "Presiona boton",
    "para leer"
};
```

4to paso Variables de control

el programa usa para **recordar en qué estado está el proyecto.**

cuántas veces se apretó el botón
qué verso corresponde mostrar
cuál es el estado actual del botón
qué valor está entregando el potenciómetro
si el LED está encendido o apagado
en qué etapa del proyecto estamos

// 4. VARIABLES DE CONTROL

```
int etapa = 0;
int buttonState = 0;
int lastButtonState = 0;
int versoActual = 0;
int valorPot = 0;
int letrasMostrar = 0;
    unsigned long tiempoUltimoClick = 0;
const unsigned long intervaloDobleClick = 400;
```

5to paso declaración de funciones

// 5. DECLARACION DE FUNCIONES

```
void mostrarLicencia();  
void mostrarAnimacion();  
void mostrarMensajeBoton();  
void mostrarTitulo();  
void mostrarVerso();  
void mostrarAutora();  
void revisarBoton();  
void revisarDobleClick();  
void reiniciarProyecto();
```

6to paso, armar setup()

El setup() sirve para preparar todo cuando Arduino se enciende. Se ejecuta una sola vez

// 6. SETUP

```
void setup() {  
  
    // Preparar componentes  
    pinMode(buttonPin, INPUT);  
    pinMode(ledPin, OUTPUT);  
  
    // LED comienza apagado  
    digitalWrite(ledPin, LOW);  
  
    // Iniciar pantalla OLED  
    if (!display.begin(SSD1306_SWITCHCAPVCC, SCREEN_ADDRESS)) {  
        for (;;);  
    }  
  
    // Limpiar pantalla  
    display.clearDisplay();  
    display.display();  
  
    // Comenzar desde el inicio  
    etapa = 0;
```

```
}
```

profundizar más en comentarios
ej de su uso

9. loop()

```
void loop() {  
  
    // ETAPA 0: licencia  
    if (etapa == 0) {  
  
        mostrarLicencia();  
        etapa = 1;  
    }  
  
    // ETAPA 1: animacion  
    else if (etapa == 1) {  
  
        mostrarAnimacion();  
        mostrarMensajeBoton();  
  
        etapa = 2;  
    }  
  
    // ETAPAS 2, 3 y 4:  
    // esperar presiones normales del boton  
    else if (etapa >= 2 && etapa <= 4) {  
  
        revisarBoton();  
    }  
  
    // ETAPA 5:  
    // potenciometro + doble clic para reiniciar  
    else if (etapa == 5) {  
  
        mostrarAutora();  
    }  
}
```



```
    revisarDobleClick();  
  }  
}
```

10. Funciones

Para no mezclar todo de una, conviene armarlas en el mismo orden del recorrido. Partamos con las más simples y después hacemos `revisarBoton()`, que es la más compleja.

Asumiendo que arriba ya tienes tus char llamados `licencia`, `titulo`, `versos` y `autora`, el bloque empieza así:

// 8. FUNCIONES

```
void mostrarLicencia() {
```

```
    // Primera parte
```

```
    display.clearDisplay();  
    display.setTextSize(1);  
    display.setTextColor(SSD1306_WHITE);
```

```
    display.setCursor(0, 0);  
    display.println(licencia[0]);
```

```
    display.setCursor(0, 10);  
    display.println(licencia[1]);
```

```
    display.setCursor(0, 20);  
    display.println(licencia[2]);
```

```
    display.display();  
    delay(2500);
```

```
    // Segunda parte
```

```
    display.clearDisplay();
```

```
    display.setCursor(0, 0);  
    display.println(licencia[3]);
```

```
display.setCursor(0, 10);  
display.println(licencia[4]);
```

```
display.setCursor(0, 20);  
display.println(licencia[5]);
```

```
display.display();  
delay(2500);
```

```
// Limpiar al terminar  
display.clearDisplay();  
display.display();  
}
```

```
void mostrarAnimacion() {  
  
    for (int i = 0; i < epd_bitmap_allArray_LEN; i++) {  
  
        display.clearDisplay();  
  
        display.drawBitmap(  
            0,  
            0,  
            epd_bitmap_allArray[i],  
            SCREEN_WIDTH,  
            SCREEN_HEIGHT,  
            SSD1306_WHITE  
        );  
  
        display.display();  
  
        delay(150);  
    }  
}
```

```
void mostrarMensajeBoton() {  
  
    display.clearDisplay();  
    display.setTextSize(1);
```

```
display.setTextColor(SSD1306_WHITE);

display.setCursor(10, 8);
display.println(mensajeBoton[0]);

display.setCursor(25, 18);
display.println(mensajeBoton[1]);

display.display();
}
```

```
void mostrarTitulo() {

display.clearDisplay();
display.setTextSize(1);
display.setTextColor(SSD1306_WHITE);

display.setCursor(20, 10);
display.println(titulo);

display.display();
}
```

```
void mostrarVerso() {

// LED segun el verso
if (versoActual % 2 == 0) {

digitalWrite(ledPin, HIGH); // versos 1, 3 y 5

} else {

digitalWrite(ledPin, LOW); // versos 2 y 4
}

display.clearDisplay();
display.setTextSize(1);
display.setTextColor(SSD1306_WHITE);
```

```
// VERSO 1
if (versoActual == 0) {

    display.setCursor(0, 6);
    display.println(partesVersos[0]);
    display.display();
}
```

```
// VERSO 2
else if (versoActual == 1) {

    // Parte 1
    display.setCursor(0, 0);
    display.println(partesVersos[1]);
    display.display();

    delay(2500);

    // Parte 2
    display.clearDisplay();
    display.setCursor(0, 6);
    display.println(partesVersos[2]);
    display.display();
}
```

```
// VERSO 3
else if (versoActual == 2) {

    // Parte 1
    display.setCursor(0, 6);
    display.println(partesVersos[3]);
    display.display();

    delay(2500);

    // Parte 2
    display.clearDisplay();
    display.setCursor(0, 6);
    display.println(partesVersos[4]);
    display.display();
}
```

```
// VERSO 4
else if (versoActual == 3) {

    display.setCursor(0, 0);
    display.println(partesVersos[5]);
    display.display();
}
```

```
// VERSO 5
else if (versoActual == 4) {

    display.setCursor(0, 6);
    display.println(partesVersos[6]);
    display.display();
}
}
```

```
void mostrarAutora() {

    // Leer potenciómetro
    valorPot = analogRead(potenciómetro);

    // Transformar el valor del potenciómetro
    // en cantidad de letras visibles
    letrasMostrar = map(
        valorPot,
        0,
        1023,
        0,
        strlen(autora)
    );

    display.clearDisplay();
    display.setTextSize(1);
    display.setTextColor(SSD1306_WHITE);
```

```

// Nombre de autora
display.setCursor(0, 0);

for (int i = 0; i < letrasMostrar; i++) {

    display.print(autora[i]);
}

// Instruccion abajo
display.setCursor(0, 22);
display.println(instruccionPot);

display.display();
}

void revisarBoton() {

    // Leer boton
    buttonState = digitalRead(buttonPin);

    // Detectar una nueva presion
    if (buttonState == HIGH && lastButtonState == LOW) {

        // Antirrebote
        delay(30);

        // MENSAJE → TITULO
        if (etapa == 2) {

            mostrarTitulo();

            etapa = 3;
        }

        // TITULO → VERSO 1
        else if (etapa == 3) {

```

```

    versoActual = 0;

    mostrarVerso();

    etapa = 4;
}

// AVANZAR ENTRE VERSOS
else if (etapa == 4) {

    // Si quedan versos
    if (versoActual < 4) {

        versoActual++;

        mostrarVerso();
    }

    // Si termino el verso 5
    else {

        digitalWrite(ledPin, LOW);

        etapa = 5;
    }
}

// Guardar estado anterior del boton
lastButtonState = buttonState;
}

void revisarDobleClick() {

    buttonState = digitalRead(buttonPin);

    // Detectar nueva presion
    if (buttonState == HIGH && lastButtonState == LOW) {

```

```

unsigned long tiempoActual = millis();

// Segundo clic dentro del intervalo
if (
    tiempoUltimoClick > 0 &&
    tiempoActual - tiempoUltimoClick <= intervaloDobleClick
) {

    reiniciarProyecto();

    tiempoUltimoClick = 0;
}

// Primer clic
else {

    tiempoUltimoClick = tiempoActual;
}
}

// Guardar estado anterior del boton
lastButtonState = buttonState;
}

void reiniciarProyecto() {

    // Volver al inicio
    etapa = 0;

    // Volver al primer verso
    versoActual = 0;

    // Apagar LED
    digitalWrite(ledPin, LOW);

```



```
// Reiniciar potenciómetro y autora  
valorPot = 0;  
letrasMostrar = 0;
```

```
// Reiniciar doble clic  
tiempoUltimoClick = 0;
```

```
// Actualizar estado del boton  
buttonState = digitalRead(buttonPin);  
lastButtonState = buttonState;
```

```
// Limpiar pantalla  
display.clearDisplay();  
display.display();  
}
```